Елементи развоја софтвера
Тема 02 · SOLID и Clean Architecture

Logger–Blogger

Поставка задатка

1. Задатак

Маја води блог и објављује текстове. Ана прати рад пословног софтвера и бележи информације, упозорења и грешке. Потребна им је заједничка апликација у којој свака може да додаје и прегледа своју врсту записа.

Реализовати конзолну апликацију у језику C# на платформи .NET 8. Режим **Bloger** користи објаве, а режим **Loger** евиденције. Пријава и мени су заједнички, док се сервис за рад са записима бира према режиму. Сервисе повезати преко интерфејса применом Dependency Inversion принципа.

1.1. Ток рада

1. При покретању проверити доступност складишта и припремити почетне корисничке налоге.

2. Затражити избор режима Loger или Bloger. За непознат избор поновити унос.

3. Затражити корисничко име и лозинку. При погрешним подацима приказати грешку и омогућити нов покушај. Празно корисничко име прекида пријаву.

4. После пријаве приказати мени за додавање записа, преглед записа и одјаву.

5. При додавању поставити аутора на пријављеног корисника и забележити време настанка. Сачувати запис одговарајуће врсте и приказати исход операције.

6. При прегледу приказати записе одабраног режима. Празна листа је исправан резултат. Одјава завршава сесију.

1.2. Пример употребе

Маја бира Bloger, пријављује се и додаје објаву са насловом, описом и типом FAMILY. Ана бира Loger и додаје евиденцију типа ERROR са описом неуспешног повезивања пословног софтвера. У прегледу се Маји приказују објаве, а Ани евиденције.

2. Модел података

Класе модела сместити у Domain.Models, а енумерације у Domain.Enums. Заједничке податке записа издвојити у апстрактну класу ZapisNaSajtu.

Табела 1. Класе модела

Класа	Својства
Korisnik	KorisnickoIme: string LozinkaHash: string ImePrezime: string
ZapisNaSajtu	Postavljeno: DateTime Opis: string Autor: Korisnik
Objava	Наслеђује ZapisNaSajtu. Naslov: string TipObjave: TipObjave
Evidencija	Наслеђује ZapisNaSajtu. TipEvidencije: TipEvidencije

2.1. Врсте записа

```
TipObjave: VLOG = 1, FRIENDS = 2, FAMILY = 3
TipEvidencije: INFO = 1, ERROR = 2, WARNING = 3
```

Конструктори класа Objava и Evidencija прослеђују време, опис и аутора базном конструктору, а затим постављају сопствена својства. Обезбедити и конструкторе потребне за JSON серијализацију.

2.2. Кориснички налог

Корисничко име је јединствено, без разликовања великих и малих слова. LozinkaHash садржи хеш лозинке. Лозинку у чистом облику не уписивати у модел, датотеку података или техничку евиденцију.

2.3. Приказ записа

ZapisFormatter у Presentation слоју приказује тип, аутора, опис и време записа, а за објаву и наслов. Формат датума је dd.MM.yyyy HH:mm. Текст скратити само за приказ.

3. Организација решења

Решење поделити на следеће пројекте. Domain садржи моделе, Application случајеве употребе и уговоре, а спољни слојеви конкретне начине чувања и приказа података.

Табела 2. Пројекти и одговорности

Пројекат	Садржај
Domain	Модели Korisnik, ZapisNaSajtu, Objava и Evidencija, енумерације TipObjave и TipEvidencije.
Application	Улазни захтеви, Result типови, интерфејси репозиторијума, сервиса и хеширања. AutentifikacioniServis, BlogServis и EvidencijaServis.
Database	Интерфејси складишта, TabeleBazaPodataka, провајдери, DatabaseProviderFactory и репозиторијуми.
Infrastructure	Pbkdf2PasswordHasher и FileLoggerServis.
Presentation	ConsoleMenu, IConsoleIO, SystemConsoleIO, ZapisFormatter, KorisniciSeeder, AppContainer и Program.cs.
Tests	NUnit/Moq тестови модела, сервиса, складишта и зависности слојева.

3.1. Дозвољене зависности

Domain нема референце на остале пројекте. Application зависи од Domain. Database зависи од Domain и Application, а Infrastructure од Application. Presentation повезује слојеве потребне за покретање апликације.

BlogServis и EvidencijaServis добијају репозиторијум и логер кроз конструктор, преко интерфејса. AutentifikacioniServis на исти начин добија репозиторијум корисника и хешер лозинки. Избор конкретних имплементација обавити у AppContainer-у.

3.2. Одвајање одговорности

Пословни сервиси не читају унос са конзоле и не отварају датотеке. Модели не форматирају конзолне табеле. Мени позива сервисе, репозиторијуми приступају складишту, а Infrastructure реализује техничко логовање и хеширање.

4. Апликативни сервиси

4.1. Регистрација и пријава

У Application.Contracts дефинисати PrijavaZahtev са корисничким именом и лозинком, и RegistracijaZahtev који додатно садржи име и презиме. IAutentifikacijaServis излаже методе:

```
Result<Korisnik> Prijava(PrijavaZahtev zahtev)
Result<Korisnik> Registracija(RegistracijaZahtev zahtev)
```

Registracija одбија празно корисничко име, лозинку и име и презиме, као и заузето корисничко име. Корисничко име и име и презиме очистити од спољних размака. Лозинку хеширати преко IPasswordHasher пре чувања налога.

Prijava проналази корисника и проверава лозинку. Непостојећи налог и погрешна лозинка враћају исту поруку о неисправним креденцијалима. Грешку приступа складишту проследити позиваоцу.

IPasswordHasher има методе Hash и Verify. Реализовати Pbkdf2PasswordHasher са насумичном salt вредношћу и записом у формату pbkdf2-sha256$iteracije$salt$hash.

4.2. Креирање и преглед записа

BlogServis и EvidencijaServis имплементирају исти интерфејс IZapisiServis:

```
Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis)
Result<IReadOnlyList<ZapisNaSajtu>> PregledZapisa()
```

BlogServis прихвата само Objava, а EvidencijaServis само Evidencija. Objavi најпре проверава тип записа. Погрешан тип одбија пре уписа. Исправан запис прослеђује IZapisiRepozitorijum-у, бележи исход кроз ILoggerServis и враћа резултат уписа. PregledZapisa враћа резултат читања одговарајућег репозиторијума.

4.3. Резултати операција

Користити Result и Result<T> из простора имена Shared. Позивалац проверава IsSuccess или IsFailure пре приступа Value. Грешка садржи Error.Code и Error.Message. За Failure проследити и празну вредност одговарајућег типа, на пример празан модел или празну колекцију.

5. Чување података и логовање

5.1. Складиште

TabeleBazaPodataka садржи иницијализоване листе Korisnici, Objave и Evidencije. IBazaPodataka излаже својства Tabele и IsReady, и методу bool SacuvajPromene(). IDatabaseProvider излаже Naziv, BazaPodataka и Result<Unit> PokusajKonekciju().

InMemoryBazaPodataka чува податке током рада процеса. SacuvajPromene() враћа true без уписа на диск. После поновног покретања претходни записи нису доступни.

JsonBazaPodataka учитава podaci.json при креирању и чува измене у исту датотеку помоћу Newtonsoft.Json. Ако датотека не постоји, формирати празне табеле. Ако постоји, али није исправна или није доступна за читање, поставити IsReady на false. Постојећи садржај тада не преписивати.

Измене најпре уписати у привремену датотеку, па њоме заменити циљну. Неуспех чувања вратити позиваоцу. Избор провајдера, in-memory или json, реализовати преко DatabaseProviderFactory.

5.2. Репозиторијуми

IKorisniciRepozitorijum има методе Dodaj, PronadjiPoKorisnickomImenu и VratiSve. IZapisiRepozitorijum има методе Dodaj и VratiSve. Све методе враћају Result са одговарајућим моделом или IReadOnlyList колекцијом.

Реализовати KorisniciRepozitorijum, ObjaveRepozitorijum и EvidencijeRepozitorijum. Пре приступа проверити IsReady. Репозиторијум записа одбија погрешан тип. При додавању позива SacuvajPromene(), а ако чување не успе, уклања додату ставку из меморијске листе и враћа грешку.

5.3. Техничка евиденција

ILoggerServis дефинише bool Loguj(string poruka). FileLoggerServis додаје временски означену поруку у log.txt. Враћа true ако је упис успео, а false при грешци приступа датотеци. Успешна и неуспешна објављивања бележити без брисања ранијих порука.

Evidencija је запис који корисник додаје у апликацији, док log.txt садржи техничке исходе операција. Неуспех техничког логовања не поништава већ сачувану објаву или евиденцију.

6. Повезивање и провера

6.1. Покретање

У AppContainer-у формирати провајдер складишта, репозиторијуме, хешер, сервис за пријаву, логер и оба сервиса записа. Метод ZapisiServisZa враћа сервис према изабраном режиму. ConsoleMenu чита и пише преко IConsoleIO, а стварни рад са конзолом обавља SystemConsoleIO.

Program.cs креира AppContainer и покреће KorisniciSeeder. Seeder проверава складиште и преко сервиса регистрације припрема налоге **maja** и **ana**, са почетном лозинком **demo12345**. Постојеће налоге не додаје поново. За пробне записе користити NasumicanTekst и вредности одговарајуће енумерације.

```
dotnet run --project Presentation
dotnet test
```

Команде покренути из корена решења. Провајдер се бира у Program.cs, позивом AppContainer.Create("in-memory") или AppContainer.Create("json").

6.2. Обавезне провере

Табела 3. Сценарији за проверу

Сценарио	Очекивани исход
Пријава и регистрација	Исправна пријава враћа корисника. Погрешна лозинка, празни подаци и заузето име враћају грешку.
Врста записа	BlogServis прихвата Objava, а EvidencijaServis прихвата Evidencija. Погрешан тип се одбија пре чувања.
Преглед	Враћају се записи одабране врсте. Празна листа не представља грешку.
Неуспешан упис	Враћа се грешка и повлачи управо додата ставка из меморијске листе.
Поновно покретање	JSON складиште учитава сачуване податке. In-memory складиште почиње празно.
Оштећен JSON	Складиште није спремно. Постојећа датотека остаје сачувана.
Зависности	Domain нема зависности од осталих слојева. Application не зависи од Database, Infrastructure или Presentation.